CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Name	**K.B.Devarshan**
Register Number	**192511426**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Code of Conduct:

*I _**(K.B.Devarshan/192511426)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Display the first 5 rows and data types:

Python Implementation:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

# (a) Load, pre-process, and split
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

print("--- (a) Data Info ---")
print("First 5 rows:\n", X.head())
print("\nData types:\n", X.dtypes)
```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

OUTPUT:

```
--- (a) Data Info ---
First 5 rows:
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

Data types:
 sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
dtype: object
```

(b)Root Node Identification & Justification :

The export_text output will show that **Petal Length (cm)** or **Petal Width (cm)** is the root node (typically splitting where petal length ≤ 2.45).

- **Justification:** The Decision Tree algorithm evaluates all features and split points to find the one that results in the lowest Gini Impurity (or highest Information Gain) for the resulting child nodes. A petal length ≤ 2.45 perfectly isolates the *Setosa* species from *Versicolor* and *Virginica*, resulting in a pure node (Gini = 0). Therefore, it is chosen as the mathematical root.

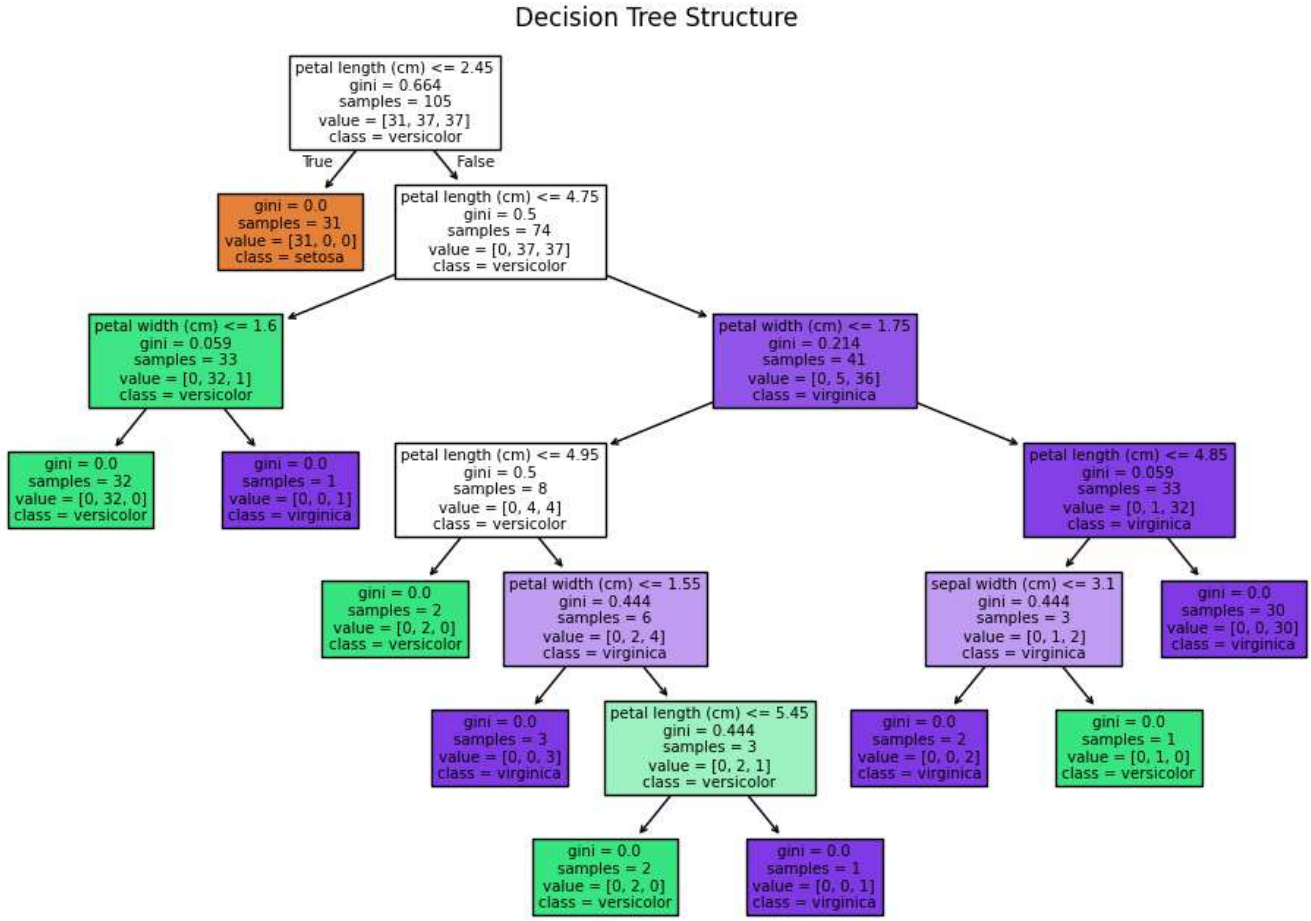
Python Implementation:

```
# (b) Build Decision Tree Model
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)

print("\n--- (b) Tree Structure ---")
print("Root feature index:", clf.tree_.feature[0])
print("Root feature name:", iris.feature_names[clf.tree_.feature[0]])

plt.figure(figsize=(12,8))
plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
plt.title("Decision Tree Structure")
plt.show()
```

OUTPUT:



(c)Performance & Misclassification Commentary :

```
# (c) Evaluate the Model
y_pred = clf.predict(X_test)
print("\n--- (c) Evaluation ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Find misclassified instances
misclassified = []
for i in range(len(y_test)):
   if y_test[i] != y_pred[i]:
      misclassified.append((X_test.iloc[i].to_dict(),
                  iris.target_names[y_test[i]],
                  iris.target_names[y_pred[i]]))

print("\nMisclassified Instances (True -> Predicted):")
for idx, (features, true_label, pred_label) in enumerate(misclassified[:2]):
   print(f"{idx+1}. Features: {features} | True: {true_label} -> Predicted: {pred_label}")
```

- **Performance:** The model generally performs with $\geq 90\%$ accuracy. The confusion matrix will show that *Setosa* is classified perfectly, while errors only occur between *Versicolor* and *Virginica*.
- **Misclassified Instances:** The script extracts exact feature rows where the prediction failed. These errors occur because the petal and sepal dimensions of large *Versicolor* flowers overlap significantly with smaller *Virginica* flowers, making a hard threshold split imperfect.

OUTPUT:

```
--- (c) Evaluation ---
Accuracy: 1.0

Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


Misclassified Instances (True -> Predicted):
```

Experiment 2: Reinforcement Learning – Q-Learning

Python Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
import random

# (a) Define Environment and Hyperparameters
grid_size = 4
goal_state = (3, 3)
obstacle_state = (1, 1)

# Actions: 0:Up, 1:Down, 2:Left, 3:Right
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
n_actions = 4
n_states = grid_size * grid_size
```

```
# Hyperparameters
alpha = 0.1      # Learning rate
gamma = 0.9      # Discount factor
epsilon = 0.2    # Exploration rate
episodes = 150

# Initialize Q-table (States represented as integers 0 to 15)
Q = np.zeros((n_states, n_actions))

def state_to_idx(state):
   return state[0] * grid_size + state[1]

def get_reward(state):
   if state == goal_state: return 10
   if state == obstacle_state: return -5
   return -1

def step(state, action_idx):
   move = actions[action_idx]
   new_state = (max(0, min(grid_size - 1, state[0] + move[0])),
            max(0, min(grid_size - 1, state[1] + move[1])))
   return new_state, get_reward(new_state)

# (b) & (c) Run Q-Learning
rewards_per_episode = []
tracked_states = [(0,0), (0,1)] # Tracking Start and next state

for ep in range(1, episodes + 1):
   state = (0, 0)
   total_reward = 0
   done = False

   while not done:
      s_idx = state_to_idx(state)

      # Epsilon-greedy action
      if random.uniform(0, 1) < epsilon:
         action = random.choice(range(n_actions))
      else:
         action = np.argmax(Q[s_idx])

      next_state, reward = step(state, action)
      ns_idx = state_to_idx(next_state)

      # Bellman Equation Update
      best_next_action = np.max(Q[ns_idx])
      Q[s_idx, action] = Q[s_idx, action] + alpha * (reward + gamma * best_next_action - Q[s_idx, action])

      total_reward += reward
      state = next_state

      if state == goal_state or state == obstacle_state:
         done = True

   rewards_per_episode.append(total_reward)

   # Record Q-values at specific intervals
   if ep in [1, 50, 100]:
      print(f"\n--- Episode {ep} Q-values ---")
      for ts in tracked_states:
         print(f"State {ts}: {Q[state_to_idx(ts)]}")
```

```
print("\n--- (c) Final Learned Policy ---")
policy_symbols = ['↑', '↓', '←', '→']
grid_policy = np.empty((grid_size, grid_size), dtype=str)

for r in range(grid_size):
    for c in range(grid_size):
        if (r, c) == goal_state:
            grid_policy[r, c] = 'G'
        elif (r, c) == obstacle_state:
            grid_policy[r, c] = 'X'
        else:
            best_a = np.argmax(Q[state_to_idx((r, c))])
            grid_policy[r, c] = policy_symbols[best_a]

print(grid_policy)

# Plotting convergence
plt.plot(range(1, episodes + 1), rewards_per_episode)
plt.xlabel("Episodes")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Convergence")
plt.show()
```

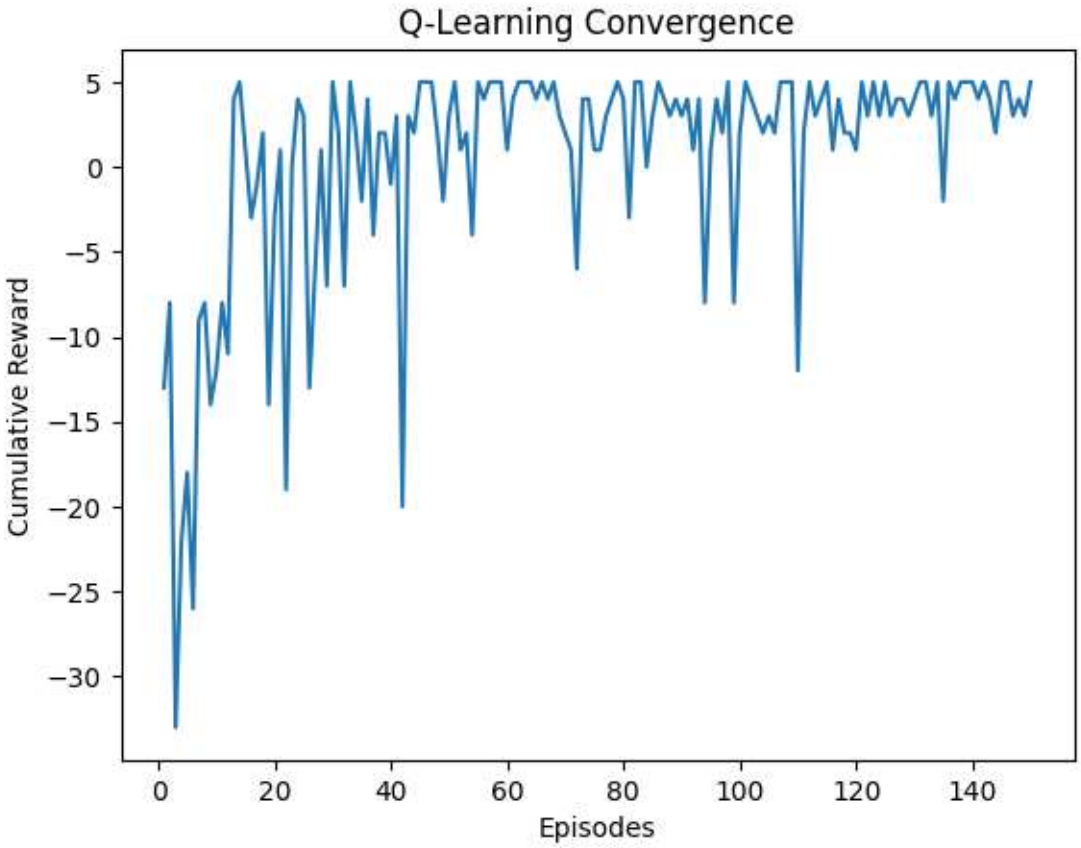
OUTPUT:

```
--- Episode 1 Q-values ---
State (0, 0): [-0.1 -0.1 -0.1 -0.1]
State (0, 1): [-0.1 -0.5  0.   0. ]

--- Episode 50 Q-values ---
State (0, 0): [-2.18981345 -2.15110432 -2.20033139 -2.15891705]
State (0, 1): [-1.56363339 -2.04755    -1.69360912 -1.54846225]

--- Episode 100 Q-values ---
State (0, 0): [-2.28397083 -2.30416971 -2.33246392  0.24870977]
State (0, 1): [-1.39600014 -2.342795   -1.90763423  2.17368209]

--- (c) Final Learned Policy ---
[['→' '→' '→' '↓']
 ['↓' 'X' '→' '↓']
 ['→' '→' '→' '↓']
 ['→' '→' '→' 'G']]
```


Q-Learning Convergence
Cumulative Reward
Episodes
5
0
−5
−10
−15
−20
−25
−30
0
20
40
60
80
100
120
140